List

PAGE NO.....
DATE / /

→ Mutable, Ordered, Heterogeneous, indexed, dynamic

1) Accessing List items.

a) Positive Indexing

```
fruits = ["apple", "banana", "cherry"]
print(fruit[0])  # output: apple.
print(fruit[2])  # Output: cherry.
```

b) Negative Indexing

```
print(fruit[-1])  # output: cherry.
print(fruit[-2])  # output: banana.
```

c) Slicing.

```
numbers = [0, 1, 2, 3, 4, 5]
print(numbers[1:4])  # output: [1,2,3]
print(numbers[::2])  # output: [0, 2, 4]
```

2) Modifying Lists: (a.) changing an item.

```
fruits = ["apple", "banana", "cherry"]
fruits[1] = "blueberry"
print(fruits)  # output: ['apple', 'blueberry', 'cherry']
```

(b) Adding items.

- append(): Add an items to the end of the list.
- insert(): Insert an item at a specific index.
- extend(): Add multiple items (from another list) to the end.

```
fruit.append("orange")
fruit.insert(1, "mango").
fruit.extend(["grape", "kiwi"])
```

DATE / /

c. Removing items

- remove() : Removes the 1st occurence of a value.
- pop() : Removes an item at a specific index. (or the last item if no index is provided.)
- del : Deletes an items or a slice of items.
- clear() : Removes all items from the list.

```
fruit.remove("blue berry").
fruit.pop(1).
del fruits[2]
fruit.clear()
```

9) List Methods

Method	Descriptions.
append(x)	Add x to the end of the list
insert(i, x)	insert x at index i.
extend(iter)	Add all items from iter (another list) to the end of the list.
remove(x)	Remove the first occurence of x
pop([i])	Removes and returns the item at index i.
clear()	Remove all items from the list.
index(x)	Returns the index of the 1st occurrence of x.

count(x)	Returns the number of times x appears in the list.
sort()	sort the list in place.
reverse()	Reverses the list in place
copy()	Returns a shallow copy of the list.

4) List Operations.

a) Concatenations.

```
list1 = [1,2]
list2 = [3,4]
combined = list1 + list2  # output [1,2,3,4]
```

b) Repetition.

```
repeated = [1,2] * 3  # output: [1,2,1,2,1,2]
```

c) Membership

```
fruits = ["apple", "banana", "cherry"]
print("banana" in fruits) # output: True
```

List Comprehension

PAGE NO.....
DATE / /

Syntax

[expression for item in iterable if condition]

- expression : The operation to perform on each time.
- item : The variable representing each element in the iterable.
- iterable : The collection of items to iterate over.
- if condition (optional) A filter to include only items that satisfy the conditions.

1) Sq. of Numbers

```
squares = [x**2 for x in range(5)]
print(squares)
```

2) Uppercase strings

```
uppercase_words = [word.upper() for word in words]
print(uppercase_words)
```

where words = ["Hello", "world", "python"]

(List Comprehension with conditions)

3) Even numbers

```
evens = [x for x in range(10) if x%2 == 0]
print(evens)
```

4) words longer than 5 characters → confusion

```
long_words = [word for word in words if len(word)>5]
print(long_words)
```

PAGE NO.....
DATE / /

5). Nested List Comprehensions.

```
matrix = [
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
]
flattened = [num for row in matrix for num in row]
print (flattened)
```

6) Extract keys from a Dictionary.

```
my_dict = {"a": 1, "b": 2, "c": 3}
keys = {key for key, value in my_dict.items()}
print (keys)
```

7) Replace Negative Numbers

```
numbers = [-1, 2, -3, 4]
non_negative = [x if x >= 0 else 0 for x in numbers]
print (non_negative)
```

Tuples

PAGE NO.....
DATE / /

→ Immutable, Ordered, Heterogeneous, Indexed, Hashable.

1) convert a list into a tuple.

```
numbers = tuple([1,2,3])
```

2) Single-item Tuple.

```
single_item = ("apple",) # Note the comma.
```

3) Tuple without Parantheses.

```
my_tuple = 1,2,3 # This is also a tuple.
```

4) Indexing

a) Positive Indexing

```
fruits = ("apple", "banana", "cherry").
print(fruits[0]) # output: apple.
print(fruits[2]) # output: cherry.
```

b) Negative Indexing

```
print(fruits[-1]) # output: cherry.
print(fruits[-2]) # output: banana.
```

5) Slicing.

```
numbers = (0,1,2,3,4,5).
print(numbers[1:4]) # output: (1,2,3).
print(numbers[::2]) # output (0,2,4).
```

PAGE NO.....
DATE / /

5) Immutability of Tuple.

fruits = ("apple", "banana", "cherry").

fruits[1] = "blueberry" # type error.

Modify

new_fruits = fruits[:1] + ("blueberry",) + fruits[2:]

print(new_fruits).

6) Tuple Operations.

a) concatenation.

tuple1 = (1, 2, 3).

tuple2 = (4, 5, 6).

combined = tuple1 + tuple2.

b) Repetition.

repeated = (1,2) * 3 # output: (1, 2, 1, 2, 1, 2).

c) Membership.

print("banana" in fruits) # output: True

PAGE NO.....
DATE / /

7) Multi-dimensional tuple.

```
matrix = (
          0   1   2
    0 ( 1 , 2 , 3 ),
    1 ( 4 , 5 , 6 ),
    2 ( 7 , 8 , 9 ),
)
print (matrix[1][2]) #output : 6 (second row, third column)
```

8) Unpacking Tuples

```
fruits = ('apple', 'banana', 'cherry').
a, b, c = fruits.
print (a)  # output : apple.
print (b)  # output : banana.
print (c)  # output : cherry.
```

9) Extended Unpacking

```
a, *b = fruits.
print (a)  # output : apple.
print (b)  # output : ['banana', 'cherry'].
```

10) Tuple methods:

Method.	Description
count (x)	Return the no. of time 'x' appears in the tuple.
index (x)	Return the index of the first occurrence of x.

PAGE NO.....
DATE / /

Sets

→ Uniqueness, Unordered, Mutable, Efficient Membership Test, Heterogeneous

my_set = {1, 2, 3, "Hello"}

1) Using the set() Constructor.

numbers = set([1, 2, 3, 2]) # Duplicates are removed.

print(numbers) # output: {1, 2, 3}.

2) Empty Set.

empty_set = set()

3) Accessing set items.

→ cannot access items by their index.

→ check if an item exists in a set using the in keyword.

fruits = {"apple", "banana", "cherry"}.

print("banana" in fruits).

print("mango" in fruits)

4) Modifying Sets.

(a) Adding items

fruits = {"apple", "banana"}.

fruits.add("cherry") # add a single item.

fruits.update(["orange", "mango"]) # add multiple items.

print(fruits)

PAGE NO.....
DATE / /

(b) Removing items

fruits.remove("banana") # Remove 'banana'
fruits.discard("mango") # No error if 'mango' doesn't exists.

fruits.pop()
fruits.clear().

(5) Set Operations.

a) Union (| or union()).

→ combines two sets, removing duplicates.

set1 = {1, 2, 3}.
set2 = {3, 4, 5}.
union_set = set1 | set2 #output: {1, 2, 3, 4, 5}.

b) intersection (& or intersection()).

intersection_set = set1 & set2 # output: {3}.

c) Difference. (- or difference()).

difference_set = set1 - set2 # output: {1, 2}

d) Symmetric Difference (^ or symmetric_difference())

→ returns items in either set but not in both.

6) Frozen sets

→ is an immutable version of a set
→ is created using the frozenset() constructor
→ can be used as keys in dictionaries or elements of other sets.

Frozen = Frozenset([1, 2, 3])

PAGE NO.....
DATE / /

Methods

Add(x)

update(iterables).

remove(x)

pop()

clear().

union(other_set).

intersection(other_set).

difference(other_set).

symmetric_difference(other_set).

issubset(other_set).

issuperset(other_set).

isdisjoint(other_set).

PAGE NO.....
DATE / /

Dictionaries.

→ Key-Value Pairs, Unordered, Mutable, Fast lookups, Heterogeneous.

1) Using the dict() Constructor

```
person = dict(name="Alice", age=25, city="New York").
```

2) From a list of Tuples.

```
person = dict([("name", "Alice"), ("age", 25), ("city", "New York")])
```

4) Empty Dictionary.

```
empty_dict = {}.
```

5) Accessing Dictionary Items.

a) Using Square Brackets

```
print(person["name"]) # output: Alice.
```

b) Using the get() Method.

```
print(person.get("name")) # output: Alice.
print(person.get("country")) # output: None.
print(person.get("country", "USA"))
```

output: USA (default value).

6) Modifying Dictionaries.

(a) Adding or Updating items.

```
person["country"] = "USA"   # add a new key-value pair
person["age"] = 26          # update an existing key.
```

PAGE NO.....
DATE / /

(b) Removing.

```
person.pop("city").      # Remove the key 'city'
person.popitem().        # Remove and return the last inserted key-value pair
del person["age"]        # Remove the key 'age'
person.clear()           # Remove all items.
```

7) Dictionary Operations. (a) Membership Test.

```
print("name" in person) # output: True.
```

(b) Length

```
print(len(person)) # output: 3.
```

(c) Iterating Over a Dictionary

```
for key in person:
    print(key, person[key]).

for value in person.values():
    print(value).

for key, value in person.items():
    print(key, value).
```

8) Nested Dictionaries.

```
people = {
    "Alice": {"age": 25, "city": "New York"},
    "Bob": {"age": 30, "city": "Los Angeles"},
}.
```

PAGE NO.....
DATE / /

Methods

keys(), values(), items(); get(key, default).
pop(key, default), popitem(), update(other_dict).
clear(), copy().

PAGE NO.....
DATE / /

Dictionary Comprehensions.

{ key : value for item in iterable if conditions }.

1) Sq. of Numbers.

```
squares = { n : n **2 for n in range(5) }.
print(squares)
#output { 0:0, 1:1, 2:4, 3:9, 4:16 }.
```

2) Upper case keys.

```
words = ["apple", "banana", "cherry"]
upper_case_dict = { word : word.upper() for
                    word in words }.
print(upper_case_dict).
#output : { 'apple' : 'APPLE', 'banana' : 'BANANA',
            'cherry' : 'CHERRY' }.
```

3) with condition

1) with even.

```
even_squares = { n : n **2 for n in range
                 (10) if n %.2 == 0 },
print(even_squares)
#output : { 0:0 , 2:4, 4:16 , 6:36, 8:64 }.
```

2) word longer than 5 characters.

```
long_words = { word : len(word) for word in words
               if len(word) > 5 }.
print(long_words).
#output : { 'banana':6, 'cherry':6 }.
```

PAGE NO.....
DATE / /

4) Matrix to Dict-

```
matrix = [
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
]
matrix_dict = {(i, j): matrix[i][j] for i in range(3)
                for j in range(3)}.
print(matrix_dict).

# output: {(0,0):1, (0,1):2, (0,2):3, (1,0):4,
           (1,1):5, (1,2):6, (2,0):7, (2,1):8,
           (2,2):9}.
```

5) Applying function to Each Value.

```
numbers = [-1, -2, 3, -4]
abs_dict = {x: abs(x) for x in numbers}.
print(abs_dict)
# output: {-1:1, -2:2, 3:3, -4:4}.
```

6) Optional using a for Loop

```
squares = {}.
for x in range(5):
    squares[x] = x**2
print(squares)
# output: {0:0, 1:1, 2:4, 3:9, 4:16}
```

4) Matrix to Dict-

```
matrix = [
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
]
matrix_dict = {(i,j): matrix[i][j] for i in range(3)
                for j in range(3)}.
print(matrix_dict).
# output: {(0,0):1, (0,1):2, (0,2):3, (1,0):4,
           (1,1):5, (1,2):6, (2,0):7, (2,1):8,
           (2,2):9}.
```

5) Applying function to Each Value.

```
numbers = [-1, -2, 3, -4]
abs_dict = {x: abs(x) for x in numbers}.
print(abs_dict)
# output: {-1:1, -2:2, 3:3, -4:4}.
```

6) Optional using a for Loop

```
squares = {}.
for x in range(5):
    squares[x] = x**2
print(squares)
# output: {0:0, 1:1, 2:4, 3:9, 4:16}.
```